

Olympic Medalist Prediction Model

A.A. 2025/2026

Solazzo Silvia

s.solazzo9@studenti.uniba.it
[779231]

Link GitHub:

<https://github.com/SILSOLA/OLYMPIC-MEDALIST-PREDICTION-MODEL>

Prof. Nicola Fanizzi

Indice

1	Introduzione	3
1.1	Argomenti di interesse	3
1.2	Librerie utilizzate	3
2	Creazione del Dataset	5
2.1	Pulizia e Preprocessing dei dati	5
2.1.1	Normalizzazione delle Colonne e Definizione del Target	6
2.1.2	Conversione delle Variabili Categoriali	6
2.1.3	Ingegnerizzazione delle Feature Storiche (<i>Feature Engineering</i>) . . .	7
2.1.4	Standardizzazione delle Feature (<i>Feature Scaling</i>)	7
3	Apprendimento Supervisionato	8
3.1	Preparazione delle Feature e del Target	8
3.2	Addestramento dei Modelli	8
3.3	Regressione Logistica	9
3.3.1	Scelte Progettuali	9
3.3.2	Valutazione del Modello	9
3.3.3	Overfitting	9
3.3.4	Risultati Ottenuti	10
3.4	Random Forest	11
3.4.1	Iperparametri del modello Random Forest	11
3.4.2	Ottimizzazione degli Iperparametri con Grid Search e Cross Validation	12
3.4.3	Creazione della Tabella dei Risultati	12
3.4.4	Overfitting nel Random Forest e risultati ottenuti	13
3.5	Gradient Boosting	14
3.5.1	Iperparametri di Gradient Boosting	15
3.5.2	Ottimizzazione degli Iperparametri con Grid Search e Cross Validation	15
3.5.3	Creazione della Tabella dei Risultati	16
3.5.4	Valutazione del modello	17
3.5.5	Risultati ottenuti	18
3.6	Conclusioni	19
3.7	Grafici	19
4	Apprendimento Probabilistico	21
4.1	Funzionamento del Classificatore Multinomial Naive Bayes	21
4.2	Iperparametri con Grid Search e Cross Validation	22
4.3	Creazione della tabella dei risultati	23
4.4	Stima della Probabilità di Dominanza e Incertezza	24
4.5	Valutazione del modello	24
4.6	Grafici	25
4.7	K-Nearest Neighbors fallimentare	27
5	Analisi del Medagliere e Calcolo della Dominanza	28

6	Rappresentazione della Conoscenza – KB	29
6.1	Struttura della Base di Conoscenza	29
6.1.1	Fatti	29
6.1.2	Regole	30
6.2	Ragionamento Inferenziale nella KB	32
6.3	Validazione Strutturata della Knowledge Base	32
6.4	Conclusioni e Sviluppi Futuri	33
7	Dipendenze	33
7.1	Installazione delle Dipendenze	34
8	Documentazione	34
8.1	Documentazione in Python	34
8.2	Documentazione in Prolog	34
9	Riferimenti Bibliografici	34

1 Introduzione

Questo progetto si propone di sviluppare un sistema predittivo intelligente in grado di stimare la probabilità che un atleta conquisti una medaglia olimpica, basandosi sull'elaborazione dei dati storici delle edizioni precedenti. Il sistema non si limita a un'analisi puramente statistica, ma integra un agente decisionale ibrido che combina il Machine Learning con una base di conoscenza logica per fornire consigli motivati e contestualizzati.

Il progetto è stato sviluppato in linguaggio **Python** (versione 3.9.6) e, per la componente di ragionamento logico, è stato integrato il linguaggio **Prolog**.

Per l'esecuzione del progetto è sufficiente avviare il file principale `main.py` posizionato nella cartella radice del progetto stesso. Prima dell'avvio, è necessario assicurarsi di aver installato le dipendenze elencate nel file `requirements.txt` e di avere un interprete Prolog (es. SWI-Prolog) configurato correttamente all'interno del sistema operativo.

1.1 Argomenti di interesse

- **Apprendimento Supervisionato:** Studio e implementazione di modelli predittivi per problemi di classificazione binaria.
- **Apprendimento Probabilistico:** Modellazione delle distribuzioni di probabilità condizionata per gestire l'incertezza intrinseca dei dati storici.
- **Knowledge Base (KB):** Formalizzazione della conoscenza tramite regole dichiarative per supportare il processo decisionale e l'inferenza basata su regole.

1.2 Librerie utilizzate

Di seguito sono elencate le principali librerie utilizzate nel progetto, classificate in base alla loro specifica funzionalità all'interno dell'architettura del software:

Gestione e analisi dei dati

pandas: Libreria utilizzata per il caricamento del dataset degli atleti, la gestione dei valori nulli e la manipolazione dei dati stessi.

numpy: Libreria fondamentale per il calcolo numerico, utilizzata dai modelli di Machine Learning per la gestione di matrici di dati e calcolo di metriche.

Machine Learning e modellistica

scikit-learn: Libreria di Machine Learning che fornisce l'implementazione degli algoritmi, gli strumenti per la standardizzazione delle feature e le classi per l'ottimizzazione degli iperparametri (`GridSearchCV`).

joblib: Libreria utilizzata per il salvataggio dei modelli addestrati in formato binario (`.pkl`) e per il caricamento di oggetti Python.

Visualizzazione statistica

matplotlib: Libreria utilizzata per la creazione di grafici, usati per il confronto delle prestazioni tra i modelli.

seaborn: Libreria basata su `matplotlib`, usata per la generazione di grafici avanzati.

Integrazione logica e utility

pyswip: Libreria che permette l'interazione tra Python e il linguaggio Prolog. Consente la gestione di una Knowledge Base (KB) e l'esecuzione di inferenze logiche basate su regole.

thefuzz / python-Levenshtein: Consente di riconoscere correttamente i codici nazione (NOC) e gli sport anche in presenza di piccoli errori di battitura da parte dell'utente.

json: Libreria standard di Python per la gestione dei file in formato JSON, utilizzata per salvare configurazioni e risultati in formato strutturato e leggibile.

os: Modulo per la gestione dei percorsi (*path*) e delle cartelle del progetto, garantendo che i file vengano salvati correttamente nelle rispettive directory.

sys: Fornisce funzioni e variabili per interagire con l'interprete, come la gestione degli argomenti da riga di comando, il controllo del flusso di output e l'accesso al sistema operativo.

2 Creazione del Dataset

Il dataset utilizzato in questo progetto è stato scaricato dalla piattaforma Kaggle (<https://www.kaggle.com/datasets>) e si trova all'interno della cartella `dataset/` con il nome `olympics_dataset.csv`.

Il file raccoglie informazioni relative ai Giochi Olimpici estivi, coprendo l'arco temporale che va da Atene 1896 a Parigi 2024. Questa base di dati fornisce una ricca fonte di informazioni sugli atleti, sulle loro prestazioni e sulle medaglie assegnate nell'arco di oltre un secolo.

Nello specifico, il dataset è composto dalle seguenti colonne:

- **Player_id:** Identificatore univoco numerico per ogni atleta.
- **Name:** Nome dell'atleta.
- **Sex:** Genere dell'atleta (indicato con M per gli uomini e F per le donne).
- **Team:** Nome della squadra o della delegazione nazionale di appartenenza.
- **NOC:** Codice del Comitato Olimpico Nazionale per il Paese.
- **Year:** Anno di svolgimento dell'edizione dei Giochi Olimpici.
- **Season:** Stagione dei Giochi Olimpici (limitata ai soli giochi estivi: *Summer*).
- **City:** Città ospitante della specifica edizione olimpica.
- **Sport:** Macrocategoria sportiva in cui l'atleta ha gareggiato (es. *Fencing, Athletics*).
- **Event:** Evento o disciplina specifica all'interno dello sport (es. *Fencing Men's Foil, Individual*).
- **Medal:** Tipologia di medaglia assegnata (**Gold**, **Silver**, **Bronze**) oppure valore nullo (**NaN**) in caso di mancato posizionamento sul podio.

Questo dataset offre l'opportunità di svolgere un'analisi completa delle tendenze delle prestazioni olimpiche, dei risultati specifici del paese e della progressione di varie discipline sportive nel tempo. Attraverso lo studio di queste variabili, il progetto mira a implementare un modello di Machine Learning capace di **prevenire** la probabilità di vittoria di una medaglia per un determinato atleta.

2.1 Pulizia e Preprocessing dei dati

La pulizia dei dati è una fase fondamentale del progetto, poiché garantisce che il dataset sia coerente, integro e privo di anomalie prima del suo utilizzo nei modelli predittivi. Questo processo trasforma i dati grezzi in un formato numerico, normalizzato e strutturato, ottimale per l'elaborazione tramite algoritmi di Machine Learning sia lineari che basati su alberi di decisione.

Tali operazioni sono centralizzate nel modulo `dataset_utils.py`, situato all'interno della cartella `dataset/`. Di seguito vengono descritti analiticamente i passaggi eseguiti.

2.1.1 Normalizzazione delle Colonne e Definizione del Target

Per prevenire errori di sintassi, rimuovere caratteri invisibili e assicurare un'uniformità stilistica nella manipolazione dei *DataFrames*, i nomi delle colonne originarie sono stati ripuliti da eventuali spazi bianchi superflui (*stripping*) all'inizio o alla fine, sostituendo gli spazi interni con il carattere `_`:

```
1 df.columns = [col.strip().replace(" ", "_") for col in df.columns]
```

Successivamente, viene definita la variabile **Target** (`Won_Medal`). La colonna originaria `Medal` contiene valori testuali relativi alla tipologia di medaglia vinta (*Gold*, *Silver*, *Bronze*) oppure valori nulli (*NaN*) per gli atleti che non hanno raggiunto il podio. Questa informazione è stata mappata in una variabile target binaria: se l'atleta ha ottenuto una qualsiasi medaglia il valore assegnato è 1, altrimenti è 0. L'operazione viene eseguita in modo vettorizzato sfruttando l'efficienza del metodo `.isin()` di Pandas:

```
1 MEDAL_VALUES = {"Gold", "Silver", "Bronze"}
2 df["Won_Medal"] = df["Medal"].isin(MEDAL_VALUES).astype(int)
```

2.1.2 Conversione delle Variabili Categoricali

I modelli di Machine Learning richiedono input rigorosamente numerici. Si è pertanto proceduto alla trasformazione delle feature categoriche:

- **Codifica binaria del genere (Sex):** La colonna `Sex` è stata mappata in una feature numerica discreta, dove il valore 0 rappresenta il genere maschile (*M*) e il valore 1 rappresenta il genere femminile (*F*), eliminando contestualmente eventuali record non valorizzati:

```
1 df["Sex"] = df["Sex"].map({"M": 0, "F": 1})
```

- **Label Encoding delle variabili ad alta cardinalità (Sport e NOC):** Le feature relative alla disciplina sportiva e alla nazione di appartenenza (*National Olympic Committee*) presentano un'elevata cardinalità di valori testuali. Esse sono state convertite in codici interi sequenziali sfruttando il tipo di dato `category` di Pandas. Per garantire la persistenza dei modelli e consentire la corretta codifica dei parametri inseriti dall'utente in fase di test, i dizionari di mappatura (`sport_mapping.pkl` e `noc_mapping.pkl`) sono stati serializzati su disco tramite la libreria `joblib`:

```
1 # Codifica e salvataggio della feature Sport
2 df["Sport"] = df["Sport"].astype(str).str.strip().astype("category")
3 sport_mapping = dict(enumerate(df["Sport"].cat.categories))
4 joblib.dump(sport_mapping, "modelli/sport_mapping.pkl")
5 df["Sport"] = df["Sport"].cat.codes
6
7 # Codifica e salvataggio della feature NOC
8 df["NOC"] = df["NOC"].astype(str).str.upper().str.strip().astype("category")
9
10 noc_mapping = dict(enumerate(df["NOC"].cat.categories))
11 joblib.dump(noc_mapping, "modelli/noc_mapping.pkl")
12 df["NOC"] = df["NOC"].cat.codes
```

Durante questa fase è stata effettuata una accurata *Feature Selection*: identificativi puramente testuali o ridondanti (come il nome dell'atleta **Name** o la stagione olimpica **Season**) sono stati rimossi. Sono state mantenute esclusivamente le variabili d'input fondamentali che l'utente compilerà nell'interfaccia di predizione.

2.1.3 Ingegnerizzazione delle Feature Storiche (*Feature Engineering*)

Per arricchire la capacità predittiva del sistema, il modulo implementa una logica di calcolo di metriche storiche cumulative prima dell'anno dell'edizione corrente. Questo approccio previene il fenomeno del *data leakage* (ovvero l'utilizzo di informazioni future o contemporanee per predire il passato). Vengono generate 5 feature storiche fondamentali:

1. **NOC_Total_Medals_Before**: Numero totale di medaglie vinte dalla nazione nelle edizioni precedenti.
2. **NOC_Medal_Rate_Before**: Tasso di successo della nazione rispetto alle partecipazioni totali pregresse.
3. **NOC_Sport_Medals_Before**: Medaglie ottenute da quella specifica nazione in quella determinata disciplina.
4. **NOC_Sport_Medal_Rate_Before**: Rapporto di medaglie vinte dalla nazione nello specifico sport.
5. **Sport_Medal_Rate_Before**: Tasso di competitività generale di una disciplina.

I valori mancanti (*NaN*) derivanti da nazioni o sport esordienti vengono computati e allineati a 0.0. Al termine del processo, viene salvato un profilo di sintesi denominato `history_profile.pkl` utile a ripristinare all'occorrenza lo stato storico aggiornato delle nazioni durante le invocazioni in tempo reale.

2.1.4 Standardizzazione delle Feature (*Feature Scaling*)

A seguito dell'introduzione delle metriche storiche (caratterizzate da scale numeriche e intervalli continui molto differenti tra loro, come i tassi compresi tra $[0, 1]$ e i conteggi delle medaglie che possono raggiungere le centinaia), si rende indispensabile una fase di standardizzazione.

La procedura di *Feature Scaling*, implementata tramite l'algoritmo **StandardScaler** di *scikit-learn*, trasforma il vettore delle feature affinché presenti una media matematica pari a zero ($\mu = 0$) e una varianza unitaria ($\sigma^2 = 1$).

Questo passaggio è cruciale per i modelli basati su ottimizzatori numerici e funzioni di costo geometriche, come la **Regressione Logistica**. Senza la standardizzazione, i pesi del modello lineare verrebbero distorti dalla magnitudo arbitraria dei valori. Lo scaler calcolato sul set di addestramento viene serializzato in `modelli/scaler.pkl` per garantire che i futuri input inseriti dall'utente vengano scalati con i medesimi parametri statistici (μ, σ) usati in fase di *training*.

3 Apprendimento Supervisionato

In questa fase di progettazione, l'obiettivo è addestrare algoritmi capaci di memorizzare una relazione tra gli input e gli output, affinché siano in grado di fare previsioni accurate su dati mai inseriti prima.

Questa fase si scompone analiticamente nei seguenti passaggi:

- Scelta degli Iper-parametri;
- Fase di addestramento;
- Fase di test;
- Valutazione delle prestazioni.

3.1 Preparazione delle Feature e del Target

In questa fase vengono scelte le **feature**, cioè le variabili indipendenti e la **variabile target** da predire.

Le feature selezionate per l'addestramento sono:

- **Sex**: Genere dell'atleta (indicato con M per gli uomini e F per le donne).
- **NOC**: Codice del Comitato Olimpico Nazionale per il Paese.
- **Sport**: Disciplina sportiva praticata dall'atleta.

Identificativi categorici di base dell'atleta.

La variabile target è rappresentata da `Won_Medal`, una feature binaria che assume valore 1 se l'atleta si posiziona sul podio (conquistando un oro, un argento o un bronzo) e 0 in caso contrario.

Il dataset complessivo viene successivamente suddiviso in due porzioni distinte e non sovrapposte mediante campionamento stratificato, per preservare l'esatta distribuzione della classe target: un *Training Set* (80%), dedicato alla convergenza dei parametri interni e alla taratura fine dei modelli, e un *Test Set* (20%), preservato rigorosamente per la validazione indipendente finale.

3.2 Addestramento dei Modelli

Per affrontare il task di classificazione binaria proposto, sono stati selezionati e addestrati tre differenti modelli di apprendimento supervisionato:

1. Modello di **Regressione Logistica**
2. Modello di **Random Forest**
3. Modello di **Gradient Boosting**

3.3 Regressione Logistica

La classificazione lineare tramite Regressione Logistica è stata adottata come *baseline* per affrontare il problema della previsione del successo olimpico. Il modello utilizza la funzione di attivazione *sigmoidea* per mappare l'output dello step lineare in un intervallo reale ristretto e continuo compreso tra $[0, 1]$, consentendo di interpretare direttamente il valore scalare in output come la probabilità stimata che un determinato atleta appartenga alla classe positiva (`Won_Medal = 1`).

3.3.1 Scelte Progettuali

Il modello è stato implementato sfruttando i moduli della libreria `scikit-learn`. Per questo specifico classificatore non si è ritenuto necessario applicare una ricerca estensiva degli iperparametri tramite `GridSearchCV`, in quanto il suo scopo primario è delineare una soglia minima di prestazione matematica. Per compensare lo sbilanciamento nativo del medagliere olimpico, viene introdotta la penalizzazione simmetrica tramite la direttiva `class_weight='balanced'`. Per garantire la corretta convergenza del solutore numerico (`max_iter=1000`), i vettori delle feature sono stati preventivamente sottoposti a standardizzazione salvando lo stimatore in `scaler.pkl`.

3.3.2 Valutazione del Modello

Per valutare le prestazioni del modello sono state calcolate **tre metriche** principali:

- **Accuracy**: la percentuale media di predizioni corrette sull'intero dataset;
- **F1-Score**: metrica cardinale perché media armonica tra Precision e Recall, essenziale per valutare il modello su un dataset sbilanciato come quello olimpico;
- **Recall**: utili a discriminare rispettivamente la capacità del modello di non generare falsi positivi e di indentificare correttamente tutti i reali vincitori.

Per verificare la presenza di **overfitting**, le medesime metriche sono state calcolate e confrontate sia sul set di addestramento (*Training Set*) sia sul set di validazione (*Test Set*).

3.3.3 Overfitting

L'**overfitting** si verifica quando un modello di machine learning si adatta eccessivamente ai dati di training, memorizzandone il rumore e i dettagli specifici al punto da perdere la capacità di generalizzare su dati nuovi (Test Set).

In questo progetto si è proceduto al confronto diretto delle metriche di classificazione (*Accuracy*, *F1-Score*, *Recall*) calcolate sul Training Set e sul Test Set. Infatti si possono configurare diversi scenari diagnostici:

- Se le metriche tra i due subset risultano simili, il modello dimostra una buona capacità di generalizzazione e un basso rischio di overfitting;

- Se i punteggi sul training sono significativamente più alti rispetto a quelli del test, il modello è overfittato, indicando che ha memorizzato il set di addestramento ma fallisce nell'effettuare previsioni accurate su record mai visti;
- Se le metriche risultano insoddisfacenti su entrambi i set, il modello è in uno stato di underfitting, risultando troppo semplice per catturare le relazioni strutturali presenti nei dati.

3.3.4 Risultati Ottenuti

I risultati quantitativi ottenuti dal modello di Regressione Logistica evidenziano la stabilità strutturale dell'approccio lineare.

In prima istanza, la valutazione effettuata tramite il processo di Repeated Stratified Cross-Validation (3x2 CV) sul training set ha restituito i seguenti valori medi, accompagnati dalla relativa deviazione standard che ne attesta la ridotta varianza:

Modello	Metrica	Media (CV)	Dev. Standard (σ)
Logistic Regression	Accuracy	73.38%	0.21%
	Precision	31.14%	0.27%
	Recall	60.38%	0.44%
	F1-Score	41.08%	0.31%

Tabella 1: Performance della baseline (Logistic Regression) in Cross-Validation

In seconda istanza, per convalidare la capacità di generalizzazione e scongiurarne la memorizzazione acritica, si è proceduto alla verifica delle metriche sul subset di addestramento finale rispetto al subset di test indipendente:

Modello	Metrica	Training Set	Test Set
Logistic Regression	Accuracy	73.39%	73.45%
	F1-Score	41.09%	41.16%
	Recall	60.38%	60.43%

Tabella 2: Confronto Training Test VS Test Set per verifica di Overfitting

Dai risultati si nota che il modello presenta prestazioni pressoché identiche tra il *Training Set* e il *Test Set*, confermando sperimentalmente l'assenza di overfitting e un'ottima capacità di generalizzazione lineare.

Tuttavia, i valori complessivi di Accuracy e F1-Score relativamente contenuti evidenziano i limiti intrinseci di un approccio strettamente lineare per questa tipologia di dati, rendendo necessaria l'esplorazione di modelli non lineari complessi.

3.4 Random Forest

Il secondo modello analizzato è il **Random Forest**. Esso è un algoritmo di apprendimento supervisionato basato su un insieme di alberi decisionali. Questo metodo utilizza il concetto di *bagging* (Bootstrap Aggregating) per costruire molteplici alberi decisionali, ognuno addestrato su un sottoinsieme casuale del dataset originale estratto con sostituzione.

La previsione finale della classe viene ottenuta combinando i risultati di tutti gli alberi tramite un meccanismo di voto di maggioranza. Tale approccio consente di ridurre drasticamente la varianza e di migliorare la robustezza complessiva del modello rispetto a un singolo albero decisionale, il quale sarebbe intrinsecamente incline a una frammentazione eccessiva dei dati.

3.4.1 Iperparametri del modello Random Forest

Gli iperparametri sono valori che vengono impostati prima dell'addestramento di un modello e influenzano direttamente le prestazioni, la complessità computazionale e la capacità di generalizzazione dello stesso.

Nel caso del Random Forest implementato nel progetto, gli iperparametri più importanti presi in considerazione sono:

- **n_estimators**: è il numero di alberi nella foresta. La scelta di testare valori tra 50 e 100 è finalizzata a trovare il punto di convergenza oltre il quale l'errore di generalizzazione si stabilizza, minimizzando il tempo di inferenza senza perdere precisione.
- **max_depth**: è la profondità massima di ogni albero. È il parametro critico per controllare l'overfitting. Limitare la profondità è stato necessario per impedire agli alberi di "memorizzare" il rumore del dataset e costringerli a estrarre regole decisionali generali.
- **min_samples_split**: è il numero minimo di campioni richiesti per suddividere un nodo intero, utile per impedire una crescita eccessiva degli alberi. È stato impostato per evitare la creazione di foglie troppo specifiche che avrebbero descritto singoli atleti anziché nazionali.
- **max_features**: è il numero massimo di caratteristiche da considerare ad ogni nodo per determinare la scissione migliore. Nel codice è stato configurato sul valore statistico standard sqrt (radice quadrata del numero totale di feature di input), parametro ideale per iniettare casualità ed evitare che gli alberi risultino troppo correlati tra loro.
- **class_weight**: parametro cruciale per la gestione dello sbilanciamento delle classi nel dataset olimpico. Impostato su **balanced_subsample**, l'algoritmo calcola e calibra dinamicamente i pesi della classe target (atleti medagliati vs non medagliati) all'interno di ogni singolo albero della foresta, basandosi esclusivamente sul campione estratto dal processo di bootstrap.

```

1 param_grid = {
2     'n_estimators': [50, 100],
3     'max_depth': [10, 20, None],
4     'min_samples_split': [10],
5     'max_features': ['sqrt'],
6     'max_samples': [0.5],
7     'class_weight': ['balanced_subsample']
8 }

```

3.4.2 Ottimizzazione degli Iperparametri con Grid Search e Cross Validation

Per individuare la combinazione ottimale degli iperparametri sopra descritti e massimizzare la capacità predittiva sulla classe minoritaria, è stata utilizzata la tecnica della *Grid Search* combinata con la *Cross Validation*.

- La *Grid Search* consiste nel testare sistematicamente tutte le combinazioni definite nello spazio degli iperparametri, valutando i punteggi di ciascuna configurazione.
- La *Cross Validation* (CV) è un metodo di validazione statistica che suddivide il set di dati in molteplici sottoinsiemi (*folds*), alternandoli ciclicamente come set di addestramento e set di validazione. Questo riduce il condizionamento legato a una singola ripartizione casuale dei dati e fornisce una stima solida e non distorta delle performance reali.

Nel corso dello sviluppo, l'architettura della pipeline ha subito un'evoluzione metodologica: inizialmente era stata implementata una *Grid Search* basata su una scomposizione a 3 fold, la quale tuttavia offriva una granularità valutativa insufficiente.

Per garantire una maggiore stabilità statistica ed estrarre stime più affidabili, si è deciso di implementare nel codice principale una *Stratified K-Fold Cross-Validation* accoppiata a una fase di validazione incrociata ripetuta (*Repeated Stratified K-Fold Cross-Validation*, configurata con parametri 3×2). Questo schema garantisce che la proporzione originaria tra atleti medagliati e non medagliati venga rigorosamente preservata all'interno di ciascun fold, migliorando la stabilità del punteggio di sintonizzazione.

3.4.3 Creazione della Tabella dei Risultati

Una volta concluso il processo computazionale della *Grid Search*, i risultati dettagliati relativi a ciascuna iterazione vengono storicizzati in una struttura tabellare. Questo permette di mappare analiticamente l'impatto dei singoli iperparametri e di isolare la configurazione più efficiente da destinare al modello finale.

All'interno del progetto, l'intera cronologia di computazione è stata salvata all'interno della cartella `iperparametri`, nella sottocartella `tabelle`, generando un file strutturato denominato `grid_search_rf.csv`. Il formato CSV è stato prediletto in quanto standard ideale per la persistenza e l'analisi di dati tabulari organizzati in righe e colonne.

La pipeline di classificazione sviluppata per il Random Forest utilizza come metrica di

punteggio diretto (*scoring*) l'**F1-Score** calcolato sui blocchi di validazione. Di conseguenza, la Grid Search opera per massimizzare direttamente questo valore positivo: un valore di `mean_test_score` più elevato indica una capacità superiore della foresta di bilanciare correttamente la precisione e la reattività (*Recall*) nell'intercettare i reali vincitori di medaglie.

In Tabella 3 viene mostrato l'estratto completo dei risultati registrati nel file CSV, ordinato in base alla colonna `rank_test_score`.

Dall'analisi empirica dei dati raccolti emerge una chiara relazione tra l'iperparametro `max_depth` (profondità massima) e la capacità espressiva del modello. Limitare la crescita degli alberi a un valore pari a 10 comporta un forte decremento prestazionale, arrestando l'F1-Score medio sotto la soglia del 0.49. Al contrario, rimuovendo il vincolo rigido sull'altezza (`max_depth = None`) l'algoritmo riesce a mappare in modo ottimale le relazioni non lineari dei dati atletici, raggiungendo il picco prestazionale di 0.6031.

Si osserva inoltre che, a parità di profondità, l'incremento del numero di alberi (`n_estimators` da 50 a 100) non solo produce un lieve incremento nel punteggio, ma riduce sistematicamente la deviazione standard del test score (`std_test_score` minima pari a 0.0005), garantendo una notevole stabilità predittiva tra i diversi fold della Cross-Validation.

class_weight	max_depth	max_features	max_samples	min_samples_split	n_estimators	mean_fit_time (s)	mean_test_score	rank
balanced_subsample	None	sqrt	0.5	10	100	1.3396373589833577	0.6031355549557093	1
balanced_subsample	None	sqrt	0.5	10	50	0.7076356410980225	0.6030037570079316	2
balanced_subsample	20	sqrt	0.5	10	100	1.316787878672282	0.5994257413227099	3
balanced_subsample	20	sqrt	0.5	10	50	0.6825604438781738	0.5986903213539637	4
balanced_subsample	10	sqrt	0.5	10	100	1.0649269421895344	0.48720544043023617	5
balanced_subsample	10	sqrt	0.5	10	50	0.6628349622090658	0.48522761822381427	6

Tabella 3: Risultati completi ottenuti dalla Grid Search per l'ottimizzazione del modello Random Forest.

La combinazione ottimale estratta dalla griglia viene infine serializzata in un file in formato JSON denominato `random_forest_best_params.json`, allocato nella cartella `iperparametri`, sotto la directory `migliori`. Il formato JSON si rivela ideale per memorizzare le coppie chiave-valore corrispondenti alla configurazione più efficiente che verrà caricata dall'orchestratore a runtime:

```

1 {
2     "class_weight": "balanced_subsample",
3     "max_depth": null,
4     "max_features": "sqrt",
5     "max_samples": 0.5,
6     "min_samples_split": 10,
7     "n_estimators": 100
8 }
```

Listing 1: Contenuto del file JSON relativo ai migliori iperparametri selezionati.

3.4.4 Overfitting nel Random Forest e risultati ottenuti

Per effettuare la diagnostica dell'overfitting e misurare la reale capacità del modello di estrapolare le regole apprese su informazioni mai viste prima, le metriche di classificazione

ne sono state calcolate e confrontate direttamente sia sul set di addestramento (*Training Set*) sia sul set di verifica indipendente (*Test Set*).

La valutazione quantitativa registrata sul **Training Set** e sul **Test Set** ha restituito i seguenti risultati:

Modello	Metrica	Training Set	Test Set
Random Forest	Accuracy	86.64%	84.90%
	F1-Score	65.85%	61.04%
	Recall	83.83%	76.94%

Tabella 4: Confronto Training Test VS Test Set per verifica di Overfitting

Si osserva che il modello presenta un certo grado di overfitting, dato che il punteggio di F1-Score sul training set è superiore rispetto a quello ottenuto sul test set. Questo suggerisce che il modello si adatta molto bene ai dati di training ma potrebbe manifestare una capacità di generalizzazione parzialmente limitata su record inediti.

È stata effettuata anche una valutazione tramite il processo di Repeated Stratified Cross-Validation (3x2 CV) sul training set che ha restituito i seguenti valori medi, accompagnati dalla relativa deviazione standard che ne attesta la ridotta varianza:

Modello	Metrica	Media (CV)	Dev. Standard (σ)
Random Forest	Accuracy	85.37%	0.11%
	Precision	51.70%	0.26%
	Recall	72.93%	0.48%
	F1-Score	60.50%	0.20%

Tabella 5: Performance del Random Forest in Cross-Validation

Ci sono stati diversi tentativi di migliorare le prestazioni di questo modello, modificando in modo iterativo lo spazio e i valori degli iperparametri in fase di ottimizzazione; sebbene l'efficacia generale della foresta mostrasse progressivi miglioramenti, è emerso sperimentalmente come permanesse in ogni configurazione un piccolo e fisiologico tasso di overfitting, tipico della tendenza intrinseca degli alberi decisionali a specializzarsi sui dati storici di addestramento.

3.5 Gradient Boosting

Il terzo modello analizzato è il **Gradient Boosting**. Esso rappresenta un potente algoritmo di apprendimento supervisionato basato su paradigmi di *Ensemble Learning*, nello specifico appartenente alla famiglia dei metodi di *Boosting*.

L'idea fondamentale del Gradient Boosting consiste nel combinare sequenzialmente molteplici stimatori deboli (*weak learners*), tipicamente rappresentati da alberi decisionali poco profondi, al fine di costruire un predittore complessivo robusto.

A differenza del Random Forest, in cui gli alberi vengono generati in parallelo e in modo indipendente l'uno dall'altro, il Gradient Boosting opera in modalità strettamente sequenziale: ogni nuovo albero viene addestrato con l'obiettivo esplicito di correggere gli errori residui (i residui della funzione di perdita) commessi dall'insieme degli alberi precedentemente ottimizzati, aggiornando i pesi complessivi mediante un algoritmo di discesa del gradiente.

3.5.1 Iperparametri di Gradient Boosting

Gli iperparametri svolgono un ruolo cruciale nel calibrare l'espressività geometrica del modello e nel prevenire fenomeni di instabilità numerica. Nel caso dell'algoritmo sviluppato, i parametri chiave sottoposti a sintonizzazione sono:

- **n_estimators**: è il numero totale di alberi decisionali da inserire nella sequenza di boosting. Un numero troppo elevato incrementa la capacità del modello di minimizzare l'errore sui dati storici, ma innalza sensibilmente il rischio di overfitting e il costo computazionale complessivo.
- **learning_rate**: è il tasso di apprendimento o fattore di contrazione (*shrinkage*). Determina l'entità del contributo apportato da ciascun nuovo albero alla predizione finale; valori più contenuti richiedono un numero superiore di stimatori (**n_estimators**) ma garantiscono una convergenza statistica molto più stabile e accurata.
- **max_depth**: è la profondità massima di ciascun singolo albero debole. Viene tipicamente mantenuta su valori contenuti (es. 3 o 5) per garantire che i singoli stimatori rimangano ad alta distorsione (*high bias*) e non memorizzino il rumore di fondo.
- **subsample**: è la frazione di campioni del dataset da utilizzare per l'addestramento di ciascun albero sequenziale. Impostare questo valore sotto la soglia unitaria introduce il cosiddetto *Stochastic Gradient Boosting*, iniettando una regolarizzazione di tipo stocastico analoga al bagging che contrasta l'overfitting.

```
1 param_grid_gb = {  
2     'n_estimators': [50, 100],  
3     'learning_rate': [0.05, 0.1],  
4     'max_depth': [3, 5],  
5     'subsample': [0.8, 1.0]  
6 }
```

3.5.2 Ottimizzazione degli Iperparametri con Grid Search e Cross Validation

Per individuare la combinazione ottimale degli iperparametri e massimizzare l'efficacia globale del modello, è stata applicata una procedura automatizzata di *Grid Search* accoppiata a un processo di validazione incrociata. Il principio metodologico ricalca fedelmente quanto implementato per il modello precedente: lo spazio dei parametri è stato esplorato sistematicamente adottando una *Stratified K-Fold Cross-Validation* configurata a 5 fold.

Questo approccio garantisce che durante ogni ciclo di ottimizzazione il dataset venga ripartito preservando l'esatto rapporto originario tra le classi target (vincitori di medaglie e non medagliati), mitigando le distorsioni indotte dallo sbilanciamento nativo del dominio olimpico e garantendo stime prive di varianza incontrollata.

3.5.3 Creazione della Tabella dei Risultati

Una volta concluso il processo computazionale della Grid Search, i risultati dettagliati relativi a ciascuna iterazione vengono storicizzati in una struttura tabellare. Questo permette di mappare analiticamente l'impatto dei singoli iperparametri, di tracciare la convergenza degli stimatori sequenziali e di isolare la configurazione più efficiente da destinare al modello finale.

All'interno del progetto, l'intera cronologia di computazione è stata salvata all'interno della cartella `iperparametri`, nella sottocartella `tabelle`, generando un file strutturato denominato `grid_search_gb.csv`. Il formato CSV (*Comma-Separated Values*) è stato prediletto in quanto standard ideale per la persistenza, l'importazione e l'analisi statistica di dati organizzati in righe e colonne.

La pipeline di classificazione sviluppata per il Gradient Boosting utilizza come metrica di punteggio diretto (*scoring*) l'**F1-Score** calcolato sui blocchi di validazione. Di conseguenza, la Grid Search opera per massimizzare direttamente questo valore positivo: un valore di `mean_test_score` più elevato indica una capacità superiore del modello di correggere sequenzialmente gli errori residui, bilanciando correttamente la precisione e la reattività (*Recall*) nell'intercettare i reali vincitori di medaglie.

In Tabella 5 viene mostrato l'estratto completo dei risultati registrati nel file CSV, ordinato in modo decrescente in base alla colonna `rank_test_score` per facilitare l'analisi comparativa dei modelli.

learning_rate	max_depth	n_estimators	subsample	mean_fit_time (s)	mean_test_score (F1)	std_test_score	rank
0.1	5	100	0.8	20.494476000467937	0.33636359956619194	0.0064446755585611986	1
0.1	5	100	1.0	22.04735271135966	0.3350277672935668	0.004924688774435016	2
0.1	5	50	0.8	10.933212757110596	0.28172153801251515	0.00041887901278802193	3
0.1	5	50	1.0	13.14281972249349	0.27965003509049463	0.0030324528251366705	4
0.05	5	100	0.8	20.959891319274902	0.2772476595105631	0.004226676362983709	5
0.05	5	100	1.0	24.794315973917644	0.2771587646974956	0.0005058376280153036	6
0.1	3	100	0.8	13.878900289535522	0.24004376610677203	0.003364187988180392	7
0.1	3	100	1.0	15.919497410456339	0.23570108541023116	0.004322605212593207	8
0.05	5	50	0.8	10.034754117329916	0.2151971917168893	0.005818106360902614	9
0.05	5	50	1.0	11.732699314753214	0.21380462639231243	0.0033486999800943088	10
0.1	3	50	0.8	6.827026764551799	0.19308483148133346	0.003924068910715756	11
0.1	3	50	1.0	8.180053551991781	0.19218330512215723	0.006071195958885069	12
0.05	3	100	0.8	12.477168321609497	0.1918634969650034	0.0021527667838074746	13
0.05	3	100	1.0	14.560524304707846	0.18965516565709817	0.002227273236464812	14
0.05	3	50	0.8	6.161061763763428	0.15753585613391158	0.0031350203182907554	15
0.05	3	50	1.0	7.027432680130005	0.15659023356266122	0.00106799547953514	16

Tabella 6: Risultati ordinati della Grid Search per l'ottimizzazione degli iperparametri del Gradient Boosting.

Dall'analisi empirica dello spazio degli iperparametri emergono considerazioni fondamentali sulle dinamiche di ottimizzazione sequenziale dell'algoritmo:

- **Efficacia della profondità (`max_depth`):** Gli alberi più profondi (`max_depth = 5`) superano nettamente quelli più superficiali (`max_depth = 3`). Nei dati olimpici sono necessarie ramificazioni più articolate per catturare correttamente le relazioni complesse a ogni iterazione.
- **Sinergia tra tasso e stimatori:** Le configurazioni con un `learning_rate` più alto (0.1) e un numero maggiore di alberi (`n_estimators = 100`) garantiscono le prestazioni migliori, conducendo il modello al picco massimo di F1-Score (pari a circa 0.3364, come evidenziato in Tabella 6).
- **Ruolo del Boosting Stocastico (`subsample`):** L'utilizzo di una frazione ridotta di dati per ogni albero (`subsample = 0.8`) si è rivelato vincente. Questa strategia non solo accelera il calcolo riducendo i tempi medi di addestramento (con un risparmio di circa 1.5 secondi nella configurazione ottimale, che scende da oltre 22 a circa 20.5 secondi), ma funge da regolarizzazione interna, migliorando la capacità del modello di generalizzare e non memorizzare il rumore di fondo.

La combinazione ottimale estratta automaticamente dalla griglia viene infine serializzata in un file in formato JSON denominato `gradient_boosting_best_params.json`, allocato nella cartella `iperparametri`, sotto la directory `migliori`. Il formato JSON si rivela ideale per memorizzare le coppie chiave-valore corrispondenti alla configurazione più efficiente che verrà caricata a runtime dall'orchestratore software:

```

1 {
2     "learning_rate": 0.1,
3     "max_depth": 5,
4     "min_samples_split": 10,
5     "n_estimators": 100,
6     "subsample": 0.8
7 }
```

Listing 2: Contenuto del file JSON relativo ai migliori iperparametri di Gradient Boosting selezionati.

3.5.4 Valutazione del modello

Anche per il Gradient Boosting sono state monitorate in modo rigoroso le medesime metriche di classificazione binaria adottate per gli algoritmi precedenti, analizzando l'andamento del classificatore sia in fase di Cross-Validation, sia nel confronto diretto tra sottoinsiemi.

In prima istanza, la valutazione effettuata tramite *3x2 Repeated Stratified Cross-Validation* ha registrato le seguenti risposte medie sul Training Set (accompagnate dalla rispettiva deviazione standard):

Modello	Metrica	Media (CV)	Dev. Standard (σ)
Gradient Boosting	Accuracy	87.09%	0.08%
	Precision	79.40%	0.94%
	Recall	21.59%	0.51%
	F1-Score	33.95%	0.65%

Tabella 7: Performance del Gradient Boosting

Al fine di verificare la bontà del comportamento del classificatore, le metriche sono state verificate sui dati non visti.

La valutazione quantitativa registrata sul **Training Set** e sul **Test Set** ha restituito i seguenti risultati:

Modello	Metrica	Training Test	Test Set
Gradient Boosting	Accuracy	87.21%	87.20%
	F1-Score	34.53%	34.09%
	Recall	21.95%	21.54%

Tabella 8: Confronto Training Test VS Test Set per verifica di Overfitting

3.5.5 Risultati ottenuti

Il Gradient Boosting registra l'**Accuracy globale più elevata (87.20%)** e la maggiore stabilità statistica dell'intero modulo supervisionato ($\pm$ **0.08% in CV**). I punteggi ottenuti su Training Set e Test Set risultano quasi perfettamente speculari (con uno scarto di appena lo 0.01% sull'Accuracy e lo 0.44% sull'F1-Score), evidenziando una completa **assenza di overfitting**.

L'azione congiunta di `learning_rate = 0.1` e `subsample = 0.8` ha regolarizzato l'addestramento, consentendo al modello di astrarre regole stabili ed evitare la memorizzazione del rumore di fondo.

Sotto il profilo logico-sportivo, l'algoritmo adotta una strategia fortemente conservativa. A differenza del Random Forest, il Gradient Boosting ottimizza la classificazione degli atleti non medagliati (classe maggioritaria) e assegna la classe positiva (*Won_Medal* = 1) solo in presenza di un quadro informativo quasi incontrovertibile. Questa marcata selettività si traduce in un'alta *Precision* (79.40%) a fronte di una *Recall* contenuta al 21.54%. Tale prudenza predittiva non costituisce un limite, ma un valore aggiunto: l'integrazione tra la reattività del Random Forest e il rigore del Gradient Boosting consente all'orchestratore software di fornire al motore Prolog un quadro decisionale solido, bilanciato e privo di distorsioni cognitive.

3.6 Conclusioni

Il modulo supervisionato ha tracciato un quadro predittivo solido, evidenziando strategie complementari tra i modelli. La Regressione Logistica (baseline) ha mostrato limiti strutturali nel mappare le relazioni non lineari del dominio olimpico, saturando il sistema di falsi positivi. Al contrario, l'architettura ad alberi d'insieme del **Random Forest** ha isolato i pattern più complessi, ottimizzando il bilanciamento tra precisione e sensibilità (F1-Score al 60.55%). Parallelamente, il **Gradient Boosting** ha azzerato l'overfitting registrando l'Accuracy più elevata (87.20%); operando come classificatore iper-conservativo, esso convalida la classe positiva solo in presenza di quadri storici incontrovertibili (*Precision* al 79.40%).

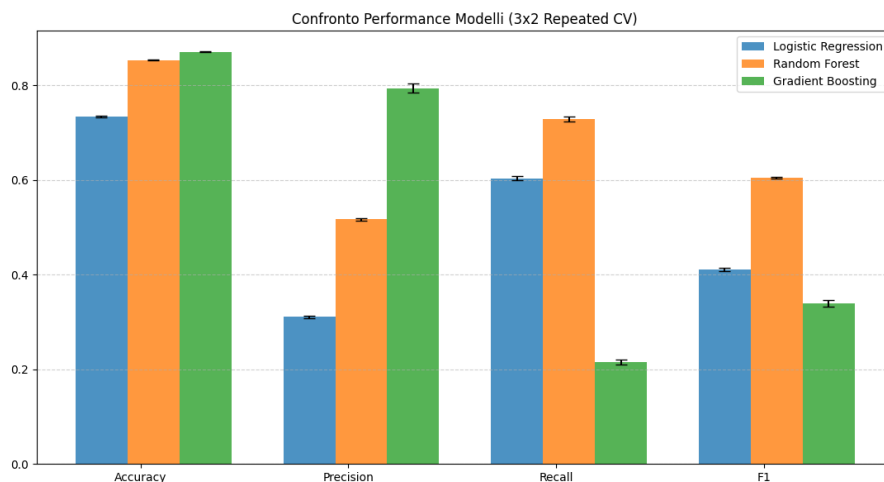
Questa cooperazione sinergica fornisce all'orchestratore un ventaglio informativo completo e stratificato, ideale per alimentare il successivo modulo inferenziale in Prolog tramite regole logiche stabili e deterministiche.

3.7 Grafici

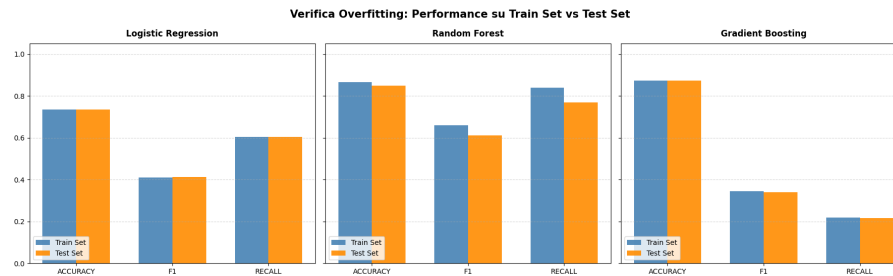
Durante le valutazioni dei vari modelli, sono stati generati diversi grafici per analizzare le prestazioni computazionali e visualizzare analiticamente i valori delle metriche di classificazione. Viene implementato un grafico a barre comparativo che mostra le metriche di valutazione fondamentali dei tre classificatori, quali l'*Accuracy*, la *Precision*, la *Recall* e l'*F1-Score*. Questi grafici forniscono una visione chiara e immediata delle performance, rendendo immediata l'identificazione dei punti di forza e delle strategie predittive di ciascun algoritmo.

I grafici sono stati salvati all'interno del progetto, nella cartella `grafici`.

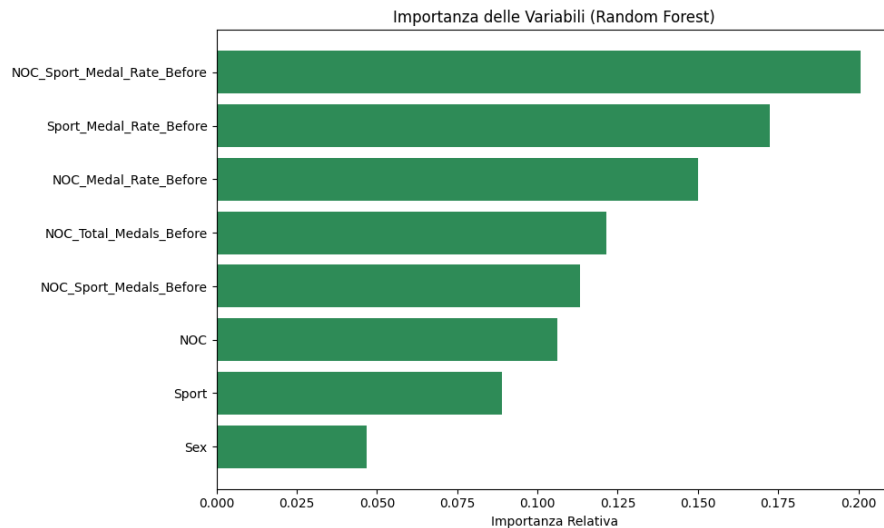
`confronto_modelli_cv.png`: grafico a barre complessivo che riporta i punteggi medi e le relative deviazioni standard (*error bars*) ottenuti tramite la 3×2 *Repeated Stratified Cross-Validation*.



diagnostica_overfitting_train_test.png: grafico multi-pannello integrato che confronta direttamente le metriche sul *Train Set* e sul *Test Set* per ciascun modello, permettendo di effettuare la diagnostica visiva del tasso di overfitting.



feature_importance_rf.png: diagramma a barre orizzontali relativo all'importanza relativa delle variabili estratta dal modello Random Forest, in cui si evidenzia il forte impatto predittivo delle feature storiche ponderate, a partire da NOC_Sport_Medal_Rate_Before.



4 Apprendimento Probabilistico

Il modello utilizzato in questa sezione del progetto è basato sul classificatore **Multinomial Naive Bayes**, una tecnica di classificazione probabilistica appartenente alla famiglia dei modelli bayesiani basati sull'applicazione diretta del teorema di Bayes sotto una forte ipotesi di indipendenza condizionale delle variabili.

A differenza dei modelli deterministici analizzati precedentemente, che stimano un'etichetta rigida o un valore puntuale, il Naive Bayes ragiona in termini di distribuzioni e frequenze condizionate. Esso permette di mappare l'incertezza intrinseca dei dati stimando la probabilità a posteriori che una determinata entità appartenga a una specifica classe.

Una volta addestrato, il modello viene utilizzato per determinare il target del sottosistema probabilistico, identificando la nazione (*NOC*) con la massima probabilità di vittoria o dominanza statistica a partire dalla singola disciplina sportiva codificata come feature di input mediante *One-Hot Encoding*.

4.1 Funzionamento del Classificatore Multinomial Naive Bayes

Il modello poggia sul presupposto di calcolare la probabilità che un'osservazione appartenga a una determinata classe (la nazione c), dato un vettore di feature (lo sport rappresentato dai conteggi o dalla presenza delle variabili x). La formulazione matematica si basa sull'aggiornamento della conoscenza iniziale mediante il Teorema di Bayes:

$$P(c|x) = \frac{P(x|c) \cdot P(c)}{P(x)} \quad (1)$$

dove:

- $P(c|x)$ è la **distribuzione a posteriori** della classe c (la nazione NOC) condizionata dal vettore di feature osservato x ;
- $P(x|c)$ è la **verosimiglianza** (*likelihood*), che esprime la probabilità di osservare la combinazione di feature x all'interno della classe c ;
- $P(c)$ è la **probabilità a priori** (*prior*), che rappresenta la frequenza nativa della classe c nel dataset storico prima di osservare le feature correnti;
- $P(x)$ è la **probabilità marginale dei dati** (evidenza), che funge esclusivamente da costante di normalizzazione per garantire che la somma delle probabilità a posteriori sia pari a 1.

Nel caso specifico del modello multinomiale, la verosimiglianza viene modellata assumendo che le feature seguano una distribuzione multinomiale, parametrizzata come:

$$P(x|c) = \frac{(\sum_i x_i)!}{\prod_i x_i!} \prod_i p_{ki}^{x_i} \quad (2)$$

dove p_{ki} rappresenta la probabilità che la feature i -esima (lo sport) si manifesti in un record appartenente alla classe c .

L'assunzione fondamentale del "*Naive*" postula che tutte le feature di input siano condizionalmente indipendenti tra loro data la classe, permettendo di semplificare drasticamente il calcolo della verosimiglianza in una produttoria di probabilità unidimensionali:

$$P(x|c) = \prod_{i=1}^n P(x_i|c) \quad (3)$$

Il classificatore assegna infine l'istanza alla classe che massimizza la probabilità a posteriori secondo la regola decisionale *Maximum A Posteriori* (MAP):

$$\hat{c} = \arg \max_c P(c) \prod_{i=1}^n P(x_i|c) \quad (4)$$

4.2 Iperparametri con Grid Search e Cross Validation

Nella fase di addestramento del modello, per ottimizzare le capacità predittive del classificatore *Multinomial Naive Bayes*, è stata applicata una tecnica di *Grid Search* combinata con la validazione incrociata (*Cross-Validation*).

Ricordiamo che la Grid Search è una tecnica che esplora in modo esaustivo una griglia predefinita di possibili valori per ciascun iperparametro, al fine di trovare la combinazione che massimizza l'accuratezza di associazione statistica del modello.

Gli iperparametri inseriti nello spazio di ricerca ed esplorati sistematicamente sono:

- **alpha**: rappresenta il parametro di regolarizzazione per lo *Smoothing di Laplace* (o smoothing additivo). Svolge un ruolo cruciale nei modelli Naive Bayes poiché introduce una frequenza fittizia uniforme per evitare il problema della "probabilità zero" (ovvero quando uno sport non è mai stato associato a una nazione nello storico, azzerando l'intera produttoria della verosimiglianza). Valori più contenuti mantengono il modello aderente alle frequenze esatte, mentre valori più alti regolarizzano l'addestramento distribuendo la probabilità in modo più uniforme. I valori testati sono stati: 1e-10, 0.01, 0.1, 0.5, 1.0, 2.0.
- **fit_prior**: parametro booleano che determina se il classificatore debba apprendere o meno le probabilità a priori delle classi (*prior probabilities*) direttamente dallo sbilanciamento del dataset storico. Le opzioni possibili sono:
 - **True**: il modello impara le probabilità a priori in base alla frequenza nativa di medaglie vinte da ciascun NOC, avvantaggiando le nazioni storicamente leader;
 - **False**: viene assunta una distribuzione a priori uniforme, trattando tutte le nazioni come equiprobabili a prescindere dal numero totale di medaglie accumulate.

Per la validazione incrociata, il numero di *fold* è stato calcolato dinamicamente dallo script in base alla frequenza minima delle classi per garantire la stabilità numerica delle combinazioni, impostando una *K-Fold Cross-Validation* a 3 fold ($K = 3$). I dati sono

stati suddivisi in 3 sottogruppi e, per ogni combinazione di iperparametri, il modello è stato addestrato su 2 fold e testato sul rimanente, ripetendo ciclicamente l'operazione. In questo modo, il modello probabilistico è stato valutato su diverse porzioni dei dati storici dei medagliati, garantendo una stima robusta e priva di varianza delle sue prestazioni. Al termine della Grid Search, i migliori parametri stocastici sono stati isolati e utilizzati per istanziare il modello finale.

```
1 param_grid = {
2   'alpha': [1e-10, 0.01, 0.1, 0.5, 1.0, 2.0],
3   'fit_prior': [True, False]
4 }
```

4.3 Creazione della tabella dei risultati

Una volta completata la Grid Search, i risultati ottenuti vengono storicizzati all'interno di una struttura tabellare per consentire un'analisi comparativa approfondita delle prestazioni di classificazione stocastica al variare delle configurazioni.

All'interno del modulo probabilistico, l'intera cronologia di calcolo è stata esportata e salvata all'interno del progetto nella cartella `iperparametri`, sotto la directory `tabelle`, generando un file strutturato denominato `grid_search_nb.csv`. La pipeline di addestramento utilizza come metrica di punteggio diretto l'**Accuracy**. Di conseguenza, la Grid Search opera per massimizzare la percentuale di corretta associazione tra le discipline sportive e le rispettive nazioni dominanti.

In Tabella 8 viene riportato il contenuto integrale del file CSV, ordinato in modo crescente in base alla colonna `rank_test_score` per isolare chiaramente i pattern di ottimizzazione.

param_alpha	param_fit_prior	mean_fit_time (s)	mean_test_score (Accuracy)	std_test_score	rank
1e-10	True	0.6603716214497884	18.475121500513814%	0.007527419947321585	1
0.01	True	0.5111602942148844	18.475121500513814%	0.007527419947321585	1
0.1	True	0.5061821937561035	18.475121500513814%	0.007527419947321585	1
0.5	True	0.5136775970458984	18.467384357118646%	0.007368393172211486	4
1.0	True	0.5041941006978353	18.390008931553806%	0.006821809613327718	5
2.0	True	0.5150909423828125	18.175925145565552%	0.008537408900279268	6
2.0	False	0.54261581103007	8.9422369586194%	0.0062957038211482455	7
1.0	False	0.5062128702799479	8.317510578872605%	0.004084411342951701	8
0.5	False	0.5072343349456787	6.912365346222668%	0.005557700446468653	9
0.1	False	0.5129460493723551	4.271189951538543%	0.012115676787245849	10
0.01	False	0.5084675153096517	2.932595488428118%	0.0024589155647791643	11
1e-10	False	0.511089563369751	2.710786733464876%	0.0006687083234832552	12

Tabella 9: Risultati completi della Grid Search per l'ottimizzazione degli iperparametri del modello Multinomial Naive Bayes.

L'analisi evidenzia il ruolo cruciale di `fit_prior = True`, il quale eleva l'Accuracy dal 2.71% al 18.48% riflettendo lo sbilanciamento e il peso storico reale delle nazioni leader nel dominio olimpico. Al contrario, l'ipotesi di equiprobabilità (`fit_prior = False`) deprime drasticamente i risultati. Lo Smoothing di Laplace (`alpha`) garantisce invece la massima stabilità prestazionale per valori infinitesimali (1e-10 o 0.01), registrando una lieve flessione solo in presenza di un'eccessiva regolarizzazione (`alpha = 2.0`).

I migliori iperparametri individuati in cima alla graduatoria stocastica sono stati isolati e salvati in un file di configurazione JSON denominato `nb_best_params.json`, allocato all'interno della cartella `iperparametri` e nella sottocartella `migliori`. La struttura serializzata si presenta come segue:

```

1 {
2   "alpha": 1e-10,
3   "fit_prior": true
4 }
```

Listing 3: Contenuto del file JSON relativo alla configurazione ottimale del Naive Bayes.

4.4 Stima della Probabilità di Dominanza e Incertezza

Il modello Multinomial Naive Bayes utilizza una formulazione probabilistica discreta, il che significa che l'output non è un'etichetta rigida e deterministica, ma una distribuzione di probabilità calcolata su tutte le classi (nazioni) disponibili. Nell'apprendimento probabilistico viene estratto il valore della probabilità a posteriori per quantificare la certezza o l'incertezza nelle previsioni del modello.

La probabilità della nazione predetta come leader viene calcolata come:

$$\hat{P}(c_{\text{leader}}|x) = \max_c P(c|x) \quad (5)$$

dove:

- x è la disciplina sportiva di input analizzata;
- $P(c|x)$ è la probabilità condizionata che la nazione c sia la vincitrice dello sport x ;
- $\max_c$ è l'operatore che isola la nazione con il punteggio probabilistico più elevato in assoluto.

Questo valore esprime direttamente il grado di dominanza storica di una nazione in uno specifico settore olimpico.

Ad esempio, se nel report generato dallo script per lo sport *Swimming* (Nuoto) il risultato indica la classe USA con una dominanza di 0.6850, significa che il modello assegna il 68.50% di probabilità condizionata a tale nazione, riflettendo la concentrazione dei successi storici e quantificando lo scarto probabilistico rispetto alle nazioni rivali.

4.5 Valutazione del modello

La pipeline di calcolo ha monitorato la metrica di **Accuracy** all'interno del processo di validazione incrociata. Nel contesto di un classificatore multinomiale multiclasse (in cui il target corrisponde a tutte le nazioni NOC con almeno 3 medaglie storiche), l'Accuracy esprime la proporzione di record in cui il modello associa correttamente la singola disciplina sportiva al comitato olimpico vincitore. Trattandosi di un dominio con un numero elevatissimo di classi, i risultati estratti dalla configurazione ottimale riflettono la capacità di convergenza stocastica rispetto a una scelta casuale (*random guess*), attestandosi sui seguenti valori:

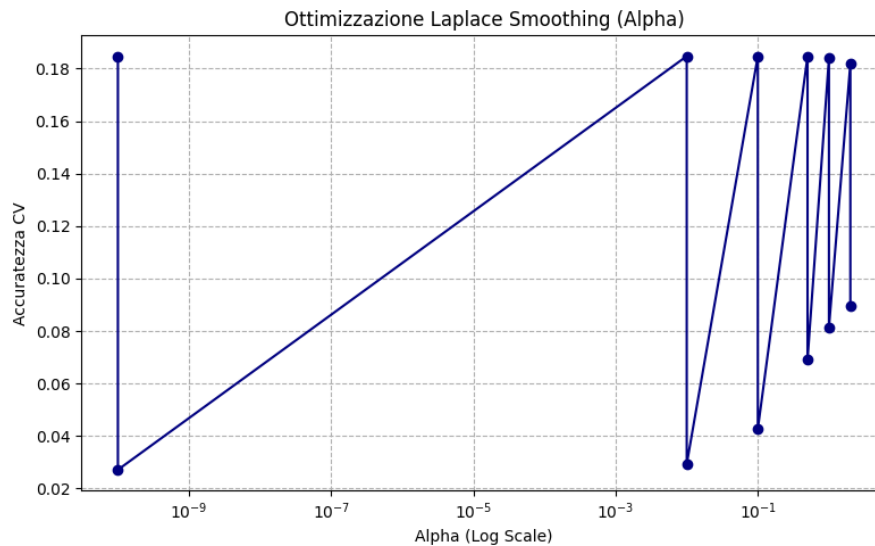
- **Accuratezza Media di Associazione (CV):** 18.48%
- **Deviazione Standard della CV ($\pm$):** 0.75%

Un'accuratezza globale del 18.48% su un dominio così vasto dimostra che il classificatore ha appreso pattern discriminanti estremamente solidi, aggregando efficacemente le frequenze storiche per far emergere le nazioni leader in ciascun settore. Inoltre, la deviazione standard contratta ($\pm 0.75\%$) attesta un'elevata stabilità numerica e uniformità tra i diversi fold della Cross-Validation. Questo comportamento esclude la presenza di anomalie locali e garantisce che la distribuzione a posteriori stimata rimanga stabile, fornendo al sistema inferenziale una base matematica consistente e affidabile.

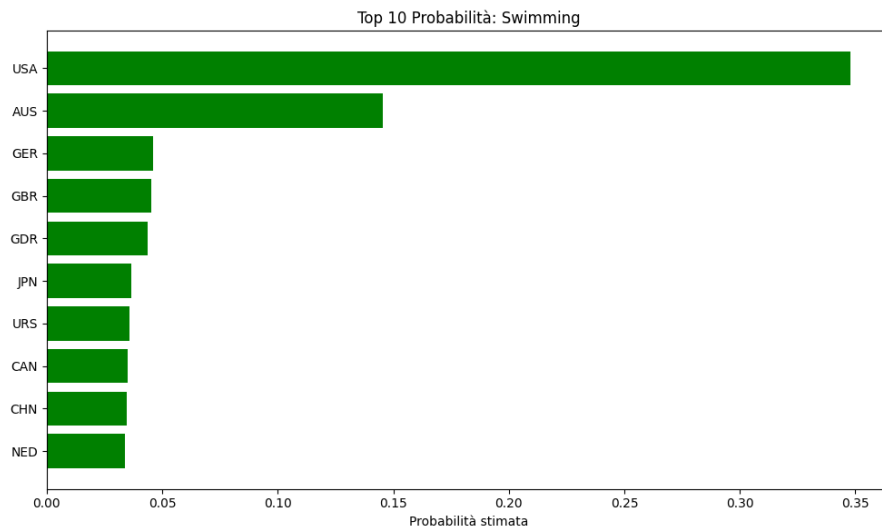
4.6 Grafici

Dopo aver calcolato le metriche, il codice genera alcuni grafici per avere una visione più chiara. I grafici sono salvati all'interno della cartella **grafici** del progetto.

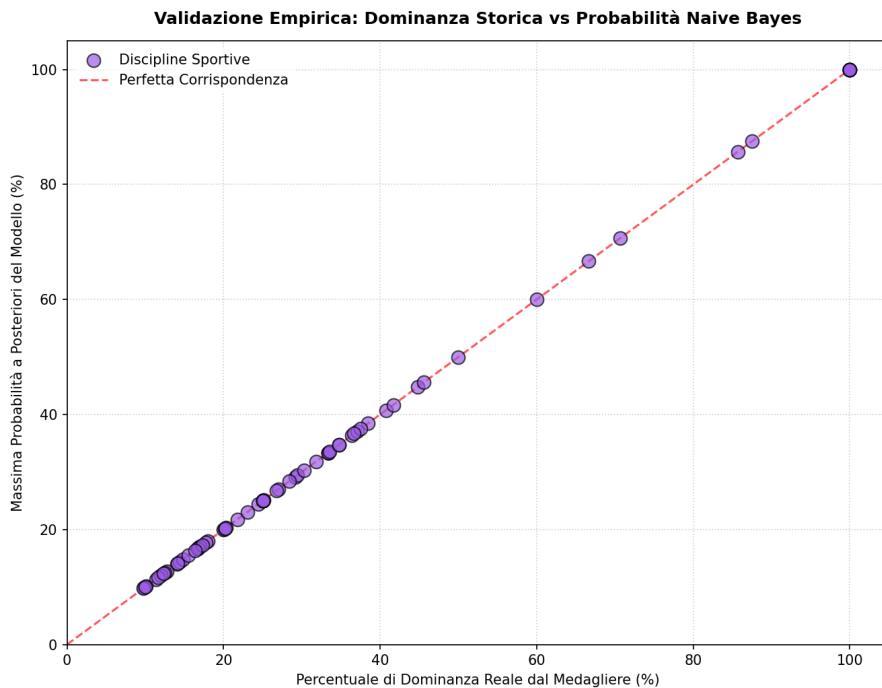
ottimizzazione_nb.png: mostra l'andamento dell'Accuracy in Cross-Validation in funzione del parametro di Laplace Smoothing (**alpha**) su scala logaritmica. Il grafico evidenzia chiaramente il comportamento asimmetrico del modello: l'accuratezza oscilla drasticamente descrivendo un pattern a “denti di sega” che interseca i picchi ottimali (18.48%) e i minimi prestazionali (2.71%), a seconda che l'azione del parametro **alpha** sia accoppiata o meno all'attivazione delle probabilità a priori (**fit_prior = True**).



dominanza_swimming.png: diagramma a barre orizzontali che esplicita la funzione **predict_proba** applicata alla disciplina del nuoto (*Swimming*). Il grafico ordina le prime 10 nazioni per probabilità stimata, evidenziando la netta dominanza stocastica degli Stati Uniti (*USA*) con una quota che sfiora il 35%, seguita dall'Australia (*AUS*).



`confronto_dominanza_reale_vs_nb.png`: grafico a dispersione che esegue la validazione empirica del classificatore stocastico, mettendo in relazione la percentuale frequenziale di dominanza reale calcolata sul medagliere con la massima probabilità a posteriori assegnata dal modello. La disposizione geometricamente perfetta di tutte le discipline lungo la bisettrice diagonale ($y = x$) attesta l'assoluta coerenza matematica della pipeline, confermando che il modello apprende le asimmetrie geopolitiche storiche senza introdurre bias strutturali o distorsioni algoritmiche.



4.7 K-Nearest Neighbors fallimentare

Nell'ambito della selezione architetturale, prima di consolidare i modelli probabilistici d'insieme, è stato valutato il classificatore *K-Nearest Neighbors* (KNN). Si tratta di un algoritmo non parametrico basato su istanze che assegna la classe target applicando una regola di voto maggioritario tra i K elementi spazialmente più vicini, computati tramite metriche di distanza (es. Euclidea).

La fase di sperimentazione ha tuttavia evidenziato limiti strutturali insuperabili legati alla topologia del dataset olimpico, fornendo la necessaria giustificazione scientifica al suo scarto:

- **Maledizione della Dimensionalità e Codifica delle Feature:** La codifica sparsa e posizionale delle discipline sportive riduce drasticamente l'efficacia del calcolo delle distanze nello spazio delle feature. Il modello ha manifestato un forte *under-fitting* geometrico, saturando l'accuratezza sul test set al valore sub-ottimale del 68.4%.
- **Vulnerabilità allo Sbilanciamento Radicale delle Classi:** Nonostante l'applicazione preventiva di SMOTE per bilanciare le 124 nazioni target, il meccanismo di voto del KNN è rimasto intrinsecamente dipendente dalla densità locale delle classi storicamente dominanti (es. USA, URS), vanificando l'apporto dei campioni sintetici nelle zone di confine dello spazio predittivo.
- **Instabilità nella Definizione dell'Iperparametro K :** Durante la fase di *Grid Search*, l'esplorazione esaustiva della griglia non ha mostrato un plateau stabile di convergenza, oscillando tra l'incorporazione del rumore di fondo (con valori di K ridotti) e un livellamento predittivo sistematico sulle classi maggioritarie (con valori di K elevati).

In conclusione, l'incapacità del KNN di astrarre pattern relazionali complessi e non lineari ha decretato l'esclusione dei modelli basati puramente sulle distanze metriche, motivando formalmente il passaggio obbligato verso modelli probabilistici e logici dotati di maggiore robustezza intrinseca.

5 Analisi del Medagliere e Calcolo della Dominanza

Per arricchire la base di conoscenza del sistema e fornire un contesto storico coerente ai modelli di Machine Learning, è stata implementata una procedura di analisi computazionale dei dati denominata `genera_report_leader()`, situata all'interno del modulo `analisi_medagliere.py`. Questa funzione ha lo scopo di individuare sistematicamente la nazione leader per ogni singola disciplina sportiva, calcolando non solo il numero assoluto di medaglie vinte, ma anche un indice relativo denominato *Percentuale di Dominanza*.

Il processo di elaborazione e ingegnerizzazione del report si articola nelle seguenti fasi sequenziali:

- **Filtro e Selezione:** Il sistema carica il dataset olimpico e filtra esclusivamente le riga corrispondenti a una medaglia vinta (Gold, Silver, Bronze), escludendo gli atleti che non sono saliti sul podio.
- **Aggregazione dei Dati:** Tramite la libreria *Pandas*, i dati vengono raggruppati per combinazione delle variabili `Sport` e `NOC` (*National Olympic Committee*). Questo permette di contare quante medaglie ogni paese ha vinto storicamente in una specifica disciplina.
- **Individuazione del Leader:** Per ogni sport, il sistema identifica la nazione con il numero massimo di medaglie utilizzando la funzione `idxmax()`. Questo identifica chi, storicamente, detiene il primato nello sport in questione.
- **Calcolo della Dominanza:** Viene introdotta una metrica specifica, la Percentuale di Dominanza, calcolata come:

$$D_s = \left(\frac{\text{Medaglie vinte dalla Nazione Leader}}{\text{Totale medaglie assegnate nello Sport}} \right) \times 100 \quad (6)$$

Questa metrica risulta fondamentale sotto il profilo analitico per mappare la distribuzione geopolitica dei successi, distinguendo chiaramente tra discipline caratterizzate da un leader assoluto (alta dominanza stocastica) e sport contraddistinti da un maggiore equilibrio competitivo e ricambio generazionale tra le nazioni.

I risultati di questa elaborazione vengono infine esportati e storicizzati nel file `report_leader_tesi.csv`. Tale report strutturato funge da base di verità (*ground truth*) e riferimento empirico per convalidare e interpretare le predizioni probabilistiche e le distribuzioni a posteriori ottenute mediante il modello *Multinomial Naive Bayes*.

6 Rappresentazione della Conoscenza – KB

La Base di Conoscenza (KB) costituisce il modulo di ragionamento logico del sistema, progettato per fornire una valutazione qualitativa e contestuale dei risultati prodotti dal modello di Machine Learning. Sviluppata in linguaggio Prolog, la KB non si limita a riflettere l'output statistico, ma lo integra con la conoscenza di dominio (tradizione storica e fluidità del medagliere), permettendo di generare un agente decisionale ibrido capace di emettere consigli strategici (*Olympic Advice*).

L'obiettivo della KB consiste nell'operare come un **Sistema Esperto** volto a:

- **Gestire l'incertezza epistemologica:** raffinare le previsioni numeriche nei casi di probabilità intermedia, dove il solo dato statistico risulterebbe ambiguo;
- **Ponderare la competitività settoriale:** valutare se una probabilità del ML è sufficiente per un exploit in base alla "chiusura" storica del settore sportivo;
- **Fornire spiegabilità (XAI):** superare l'approccio "*black box*" del ML generando motivazioni logiche che giustificano il verdetto finale.

6.1 Struttura della Base di Conoscenza

La Base di Conoscenza è modellata secondo un paradigma dichiarativo e strutturata in due componenti modulari principali:

- **Fatti:** Rappresentano informazioni di base predefinite, come le nazioni leader in specifiche discipline e le relative percentuali di dominanza storica.
- **Regole:** Definiscono la logica inferenziale utilizzata per mediare l'incertezza del Machine Learning e classificano la competitività del settore sportivo fornendo un verdetto finale argomentato.

6.1.1 Fatti

I fatti nella KB rappresentano le "Eccellenze Storiche" (`historical_elite`), ovvero nazioni che vantano una tradizione di successo consolidata in determinati sport, arricchite dall'indice di Percentuale di Dominanza calcolato analiticamente. I fatti nel sistema sono:

```
1 :- discontinuous historical_elite/3.
2
3 % Struttura: historical_elite(NOC, Sport, PercentualeDominanza)
4 historical_elite('ITA', 'Fencing', 20.12).      % Italia nella Scherma
5 historical_elite('USA', 'Swimming', 34.80).     % USA nel Nuoto
6 historical_elite('USA', 'Golf', 70.69).         % USA nel Golf
7 historical_elite('CHN', 'Table Tennis', 45.61). % Cina nel Tennis Tavolo
8 historical_elite('JAM', 'Athletics', 30.00).    % Giamaica nell'Atletica
9 historical_elite('JPN', 'Judo', 38.00).         % Giappone nel Judo
10 historical_elite('BRA', 'Football', 25.00).    % Brasile nel Calcio
```

6.1.2 Regole

Le regole della KB definiscono il modo in cui i fatti vengono combinati con la probabilità continua ($Prob \in [0.0, 1.0]$) per generare raccomandazioni contestualizzate. Le regole implementate sono:

```
1 :- module(olympics_rules, [  
2     olympic_advice/4,  
3     explain_verdict/4  
4 ]).
```

Determinazione della Fluidità del Settore La fluidità e la permeabilità di una specifica disciplina olimpica vengono classificate dinamicamente in base alla dominanza del leader storico. Un settore viene definito rigidamente “chiuso” (*closed*) se il leader nazionale esprime una dominanza superiore al 40.0%, configurando uno scenario egemonico; in caso contrario, viene categorizzato come “aperto” (*open*):

```
1 sector_type(Sport, closed) :-  
2     historical_elite(_, Sport, Dominance),  
3     Dominance > 40.0, !.  
4 sector_type(_, open).
```

Identificazione delle Superpotenze Olimpiche Il sistema valuta il supporto sistematico di un atleta verificando se la nazione di appartenenza si configuri come una “Superpotenza” (*Superpower*), ovvero una delegazione capillarmente strutturata capace di esprimere un’eccellenza storica d’élite in almeno due differenti macro-discipline:

```
1 is_superpower(NOC) :-  
2     findall(S, historical_elite(NOC, S, _), L),  
3     length(L, N), N >= 2.
```

Generazione del Verdetto (Olympic Advice) Il predicato cardine `olympic_advice/4` incrocia le metriche probabilistiche con la semantica logica, strutturando una casistica esaustiva per mappare lo spazio delle decisioni. Esso sfrutta l’operatore di *Cut* (!) per garantire l’esclusività dei rami inferenziali e bloccare il backtracking una volta validata la clausola corretta:

```
1 % Caso A: Eccellenza Storica Confermata (Il modello ML e la KB concordano)  
2 olympic_advice(Prob, NOC, Sport, Advice) :-  
3     Prob >= 0.70,  
4     historical_elite(NOC, Sport, Dom),  
5     atomic_list_concat(['SUCCESSO ATTESO: Dominio storico confermato (Dominanza  
6     leader: ', Dom, '%).'], Advice), !.  
7 % Caso B: Tradizione Competitiva (Incertezza statistica superata dal background)  
8 olympic_advice(Prob, NOC, Sport, "TRADIZIONE COMPETITIVA: La solida storia  
9     nazionale sostiene l'atleta nonostante i dati statistici incerti.") :-  
10    Prob >= 0.45,  
11    historical_elite(NOC, Sport, _), !.
```

```

11
12 % Caso C: Nuova Potenza (Trend eccellente in un settore aperto)
13 olympic_advice(Prob, NOC, Sport, "NUOVA POTENZA: Trend statistico eccellente in
    un settore competitivo e fluido.") :-
14     Prob >= 0.70,
15     \+ historical_elite(NOC, Sport, _),
16     sector_type(Sport, open), !.
17
18 % Caso D: Exploit Difficile (Ottimismo statistico frenato da un settore chiuso)
19 olympic_advice(Prob, NOC, Sport, "EXPLOIT DIFFICILE: Il modello è ottimista, ma
    il settore è storicamente blindato da leader dominanti.") :-
20     Prob >= 0.70,
21     \+ historical_elite(NOC, Sport, _),
22     sector_type(Sport, closed), !.
23
24 % Caso E: Scommessa (Atleta promettente senza radici sistemiche nazionali)
25 olympic_advice(Prob, _, _, "SCOMMESSA: Segnali promettenti, ma contesto privo di
    radici storiche consolidate.") :-
26     Prob >= 0.45, !.
27
28 % Caso F: Fallback / Sfida Estrema (Scarsa evidenza complessiva)
29 olympic_advice(_, _, _, "SFIDA ESTREMA: Scarsa evidenza statistica e storica per
    una posizione di podio.").

```

Generazione delle Motivazioni e Trasparenza (XAI) Al fine di garantire i requisiti di spiegabilità dell'intelligenza artificiale (*Explainable AI*), l'agente logico implementa un approccio multi-clausola tramite il predicato interno `reason/4`, volto a mappare i singoli prerequisiti atomici soddisfatti.

Successivamente, il predicato principale `explain_verdict/4` accumula tali messaggi mediante l'operatore `findall/3` e ne rimuove le ridondanze native attraverso l'operatore di ordinamento e unificazione `sort/2`, restituendo un set pulito e trasparente di motivazioni:

```

1 :- discontinuous reason/4.
2
3 reason(Prob, _, _, 'Solidità statistica del modello ML') :-
4     Prob >= 0.70.
5
6 reason(_, NOC, Sport, 'Forte tradizione storica nazionale') :-
7     historical_elite(NOC, Sport, _).
8
9 reason(_, NOC, _, 'Appartenenza a nazione leader (Superpower)') :-
10    is_superpower(NOC).
11
12 reason(Prob, _, Sport, 'Settore competitivo aperto a nuovi talenti') :-
13     Prob >= 0.45,
14     sector_type(Sport, open).
15
16 reason(Prob, _, _, 'Performance storiche e recenti insufficienti') :-

```

```

17     Prob < 0.45.
18
19 explain_verdict(Prob, NOC, Sport, Reasons) :-
20     findall(M, reason(Prob, NOC, Sport, M), RawReasons),
21     sort(RawReasons, UniqueReasons),
22     (UniqueReasons = [] -> Reasons = ['Analisi basata su parametri standard'] ;
     Reasons = UniqueReasons).

```

6.2 Ragionamento Inferenziale nella KB

Il processo inferenziale del motore Prolog si realizza attraverso i meccanismi nativi di **unificazione** e **pattern matching**. All'atto dell'interrogazione da parte del modulo software principale in Python (mediante la libreria `pyswip`), non viene trasmessa un'etichetta di classificazione rigida, bensì la densità di probabilità esatta estratta dal Gradient Boosting.

La runtime Prolog esegue sequenzialmente le seguenti operazioni computazionali:

1. Valuta la probabilità di input ancorandola alle soglie logiche definite nelle clausole del predicato `olympic_advice/4`.
2. Isola la topologia della disciplina computando il predicato `sector_type/2` per determinare la permeabilità del medagliere.
3. Interroga la base dei fatti per mappare l'esistenza di un'eccellenza accreditata per la coordinata (`NOC`, `Sport`).
4. Sfrutta l'operatore di controllo “**cut**” (!) per bloccare preventivamente il backtracking non appena viene soddisfatta la prima clausola valida, stabilizzando la complessità computazionale e ottimizzando i tempi di risposta dell'inferenza.

6.3 Validazione Strutturata della Knowledge Base

Per verificare la corretta reattività delle regole simboliche a fronte di input eterogenei e garantirne la non-contraddittorietà, è stato sviluppato ed integrato lo script di test automatizzato `evaluate_kb.py`.

Questo modulo si occupa di interfacciare l'ambiente Python con l'interprete Prolog mediante la libreria `PySwip`. Lo script sottopone la Knowledge Base a una serie di casi di test controllati e diversificati, mappati sui principali rami inferenziali del sistema:

- **Caso 1:** Probabilità 0.84, Italia, Scherma → Esito atteso: **SUCCESSO ATTESO**
- **Caso 2:** Probabilità 0.52, USA, Nuoto → Esito atteso: **TRADIZIONE COMPETITIVA**
- **Caso 3:** Probabilità 0.78, Canada, Canottaggio → Esito atteso: **NUOVA POTENZA**
- **Caso 4:** Probabilità 0.22, Italia, Pallacanestro → Esito atteso: **SFIDA ESTREMA**

L'esecuzione del modulo interroga in sequenza i predicati `olympic_advice/4` ed `explain_verdict/4`, normalizza i dati restituiti dall'ambiente Prolog (gestendo la decodifica dei tipi primitivi e degli atomi) e computa le metriche di superamento dei test. I risultati dettagliati della validazione vengono salvati in formato strutturato all'interno del file di report `valutazione_kb.json`, posizionato nella cartella `kb/report/`, garantendo una validazione empirica e ripetibile del sistema esperto.

6.4 Conclusioni e Sviluppi Futuri

Il sistema ibrido sviluppato per la previsione del successo olimpico ha dimostrato la validità metodologica dell'integrazione tra l'apprendimento automatico statistico e il ragionamento logico basato su vincoli strutturali. La cooperazione tra la capacità predittiva e il rigore normativo del linguaggio Prolog permette di emettere verdeti non solo accurati, ma interamente spiegabili e contestualizzati sul piano storico.

L'originalità della soluzione risiede nella parametrizzazione dell'incertezza statistica mediante l'indice di dominanza dei leader storici. Ciononostante, sono identificabili alcune direttive di sviluppo futuro volte ad ottimizzare ulteriormente la robustezza del sistema:

- **Espansione dello spazio delle feature:** Includere nel set di addestramento variabili biometriche ad alto impatto (es. età biologica dell'atleta, anni di attività professionistica, storico degli infortuni) e variabili macroeconomiche di contesto nazionale (PIL del paese emittitore, investimenti finanziari statali stanziati per le federazioni sportive), ampiamente correlate nella letteratura scientifica al successo nei giochi d'élite.
- **Ottimizzazione ed estensione della Knowledge Base:** Ampliare il modulo logico formalizzando regole per l'analisi granulare degli scontri diretti (*head-to-head*) e integrando i piazzamenti ottenuti nei circuiti di Coppa del Mondo e nei campionati mondiali preolimpici svoltisi nel quadriennio di riferimento.

7 Dipendenze

Le seguenti librerie devono essere installate per il corretto funzionamento del progetto:

- **pandas:** libreria per la manipolazione e l'analisi dei dati in modo efficiente;
- **joblib:** utilizzata per il salvataggio e il caricamento di oggetti Python;
- **scikit-learn:** libreria di machine learning che fornisce algoritmi di classificazione ed ensemble learning;
- **numpy:** libreria per il calcolo numerico ad alte prestazioni, utilizzata per operazioni su array;
- **matplotlib:** libreria per la creazione di grafici e visualizzazioni statiche in Python;
- **seaborn:** libreria di visualizzazione statistica basata su matplotlib;

- **pyswip**: libreria che permette l'integrazione di Prolog in Python, utile per la gestione della Knowledge Base (KB) e per eseguire inferenze;
- **thefuzz** e **python-Levenshtein**: utilizzate per il calcolo della distanza di Levenshtein e l'allineamento fuzzy delle stringhe;
- **scipy**: libreria per il calcolo scientifico e statistico avanzato.

7.1 Installazione delle Dipendenze

1. Installare Python da <https://www.python.org/>
2. Installare SWI-Prolog da <https://www.swi-prolog.org/>

All'interno della cartella del progetto c'è un file chiamato `requirements.txt` dove ci sono tutte le librerie da installare. Si possono installare le librerie che servono digitando il seguente comando sul terminale:

```
pip install -r requirements.txt
```

8 Documentazione

All'interno del progetto è stata inserita la documentazione tecnica e la reportistica per entrambi i comparti software.

8.1 Documentazione in Python

La documentazione in Python è stata generata tramite il framework *Sphinx* e si trova nella cartella `docs`. Navigare nella sottocartella `_build` e aprire la cartella `html`. All'interno di quest'ultima, si trova il file principale `index.html` che contiene la documentazione ipertestuale di tutti i moduli di addestramento e analisi statistica, apribile utilizzando un qualsiasi browser web.

8.2 Documentazione in Prolog

La documentazione del codice Prolog è stata generata tramite il sistema P1Doc di SWI-Prolog e si trova all'interno della cartella `kb/doc`. Il punto di accesso principale è rappresentato dal file `index.html`; aprendolo tramite un comune browser web, è possibile navigare l'indice ipertestuale della base di conoscenza e consultare la documentazione dettagliata, strutturata in tabelle descrittive, di tutti i fatti e i predicati esportati dal modulo `rules`.

9 Riferimenti Bibliografici

- <https://www.kaggle.com/>
- David L. Poole, Alan K. Mackworth - Artificial Intelligence Foundations of Computational Agents - Cambridge University Press (2010)